

Documento Final de Entrega - Tech Challenge

Data da entrega: 20/06/2026

1. Nome do grupo

Entrega individual - Yasmin Barcelos Pires

2. Participantes

Nome completo	RM	Papel no projeto
Yasmin Barcelos Pires	RM370897	Desenvolvimento individual do projeto

3. Usernames no Discord

Participante	Username no Discord
Yasmin Barcelos Pires	Não informado - substituir pelo username do Discord antes da entrega

4. Nome do projeto

AutoCare Hub

5. Link do repositório privado

<https://github.com/yxsbx/SOAT-FIAP>

6. Link da documentação DDD ou Miro

Documentação DDD versionada no repositório:

`docs/DDD_DOCUMENTATION.md`

Event Storming versionado no repositório:

`docs/EVENT_STORMING.md`

Não foi informado um quadro externo no Miro. A documentação oficial da entrega está versionada nos arquivos acima.

7. Link ou rota do Swagger

Swagger local com a API em execução:

`http://localhost:8080/swagger-ui.html`

Contrato OpenAPI versionado:

`docs/openapi/openapi.yaml`

Não há URL pública informada. Para avaliação local, usar a rota do Swagger apresentada acima.

8. Descrição resumida do projeto

O AutoCare Hub é um MVP acadêmico desenvolvido para o Tech Challenge FIAP. A solução entrega uma API REST para gestão de uma oficina mecânica, centralizando clientes, veículos, serviços, peças, estoque, Ordens de Serviço, geração de orçamento, aprovação de orçamento e acompanhamento do atendimento.

O foco principal da entrega é o backend monolítico em Java/Spring Boot, com documentação OpenAPI/Swagger, autenticação JWT, persistência relacional com PostgreSQL, migrations com Flyway, Docker, testes automatizados e relatório de vulnerabilidades. O repositório também inclui um frontend Vue/Vite demonstrativo para apoiar a apresentação da experiência web.

9. Objetivo do MVP

O objetivo do MVP é entregar um backend funcional, testável e documentado para apoiar o ciclo de atendimento de uma oficina mecânica, desde o cadastro do cliente até a entrega do veículo.

Objetivos específicos:

- Administrar clientes, veículos, serviços, peças e insumos.
- Criar Ordens de Serviço completas.
- Gerar orçamento automaticamente a partir de serviços e peças.
- Permitir aprovação do orçamento.
- Controlar status da Ordem de Serviço.
- Permitir consulta do andamento da OS pelo cliente.
- Proteger APIs administrativas com JWT.
- Validar dados sensíveis como CPF/CNPJ e placa.
- Disponibilizar documentação Swagger/OpenAPI.
- Entregar execução local reproduzível com Docker.
- Demonstrar testes automatizados e análise de vulnerabilidades.

10. Resumo das funcionalidades entregues

Funcionalidades principais entregues:

- Autenticação com JWT.
- CRUD administrativo de clientes.
- CRUD administrativo de veículos.
- CRUD administrativo de serviços da oficina.
- CRUD administrativo de peças e insumos.
- Controle de estoque com entrada, saída, reserva, liberação e baixa.
- Criação de Ordem de Serviço com cliente identificado por CPF/CNPJ.
- Cadastro ou vinculação de veículo na criação da OS.
- Inclusão de serviços solicitados na OS.
- Inclusão de peças e insumos na OS.
- Geração automática de orçamento.
- Aprovação de orçamento.
- Início controlado da execução após aprovação.
- Controle de status da OS.
- Consulta de OS pelo cliente via API.
- Métrica de tempo médio de execução de Ordens de Serviço finalizadas.
- Cadastro e consulta de interessados em parceria.
- Gestão básica de usuários, permissões e preferências.

- Documentação Swagger/OpenAPI.
- Dockerfile e docker-compose.yml.
- Testes unitários e de integração.
- Documentação DDD e Event Storming.
- Relatório de vulnerabilidades.

11. Resumo da arquitetura

O backend foi implementado como um monolito em camadas, separando regras de negócio, casos de uso, infraestrutura e interfaces REST.

Estrutura principal:

```
src/main/java/br/com/autocarehub
|-- domain
|   |-- enums
|   |-- exception
|   |-- model
|   |-- policy
|   |-- service
|   |-- valueobject
|-- application
|   |-- port
|   |-- usecase
|-- infrastructure
|   |-- config
|   |-- persistence
|   |-- security
|-- interfaces
    |-- rest
        |-- controller
        |-- exception
        |-- mapper
```

Responsabilidades:

- domain: entidades, value objects, enums, exceções, políticas e regras de negócio.
- application: use cases que coordenam os fluxos da aplicação.
- infrastructure: persistência JPA, configurações, segurança e adaptadores.
- interfaces: controllers REST, tratamento de exceções e mapeamento de DTOs.

Os controllers não acessam repositories diretamente. Eles delegam para use cases, que aplicam regras de negócio e usam portas/adaptadores para persistência.

Tecnologias principais:

- Java 21.
- Spring Boot 4.1.0.
- Spring Web MVC.
- Spring Security.
- Spring Data JPA.
- PostgreSQL 16.
- Flyway.
- Maven.
- JWT com JJWT.
- Springdoc Swagger UI.

- OpenAPI Generator.
- JaCoCo.
- OWASP Dependency-Check.
- JUnit 5, Mockito, H2 e Testcontainers.

12. Justificativa do banco de dados

O banco escolhido para o ambiente principal foi o PostgreSQL.

A escolha é adequada ao domínio porque o MVP trabalha com dados relacionais e exige consistência entre entidades:

- Um cliente pode ter vários veículos.
- Uma Ordem de Serviço pertence a um cliente e a um veículo.
- Uma Ordem de Serviço possui serviços e peças.
- Peças possuem quantidade em estoque, quantidade reservada e disponibilidade.
- Movimentações de estoque precisam manter integridade.
- Usuários podem estar associados a perfis, empresas e permissões.

O PostgreSQL oferece transações, integridade referencial, índices, maturidade operacional e boa integração com Spring Data JPA e Flyway. Nos testes automatizados, o projeto usa H2 e/ou Testcontainers conforme o tipo de teste, mantendo a execução local reproduzível.

13. Aplicação de DDD

O projeto aplica DDD de forma pragmática dentro de um monolito em camadas.

Elementos aplicados:

- Entidades de domínio: Customer, Vehicle, ServiceOrder, WorkshopService, Part, Budget, BudgetItem, StockMovement, User.
- Value Objects: Document, Plate, Money, Address.
- Enums de domínio: ServiceOrderStatus, StockMovementType, DocumentType, UserRole.
- Exceções de domínio: DomainException, InvalidServiceOrderStatusTransitionException.
- Política de domínio: PlatformFeePolicy.
- Use cases de aplicação para criação de OS, geração de orçamento, aprovação de orçamento, controle de estoque, CRUDs administrativos e autenticação.

A Ordem de Serviço é o agregado central do atendimento. Ela conecta cliente, veículo, serviços, peças, orçamento, status e histórico de andamento.

Documentação completa:

`docs/DDD_DOCUMENTATION.md`

14. Event Storming

O Event Storming foi documentado para os fluxos principais do MVP:

- Criação e acompanhamento da Ordem de Serviço.
- Gestão de peças e insumos.

O documento registra atores, comandos, eventos de domínio, agregados, políticas, regras de negócio, exceções, fluxos principais, fluxos alternativos e pontos de decisão.

No MVP, os eventos são usados como linguagem de modelagem e documentação. O sistema não implementa event store dedicado.

Documento versionado:

docs/EVENT_STORMING.md

15. Linguagem Ubíqua

Termo	Significado
Cliente	Pessoa física ou jurídica atendida pela oficina, identificada por CPF ou CNPJ.
Documento	CPF ou CNPJ validado, normalizado e usado para evitar duplicidade.
Veículo	Veículo pertencente a um cliente, identificado por placa, marca, modelo e ano.
Placa	Identificador do veículo, aceitando formato brasileiro antigo e Mercosul.
Ordem de Serviço	Registro central do atendimento da oficina.
Status da OS	Etapa atual da Ordem de Serviço.
Serviço	Atividade executável pela oficina, com descrição, preço e tempo estimado.
Peça/Insumo	Item físico usado no atendimento e controlado em estoque.
Estoque	Quantidade total, reservada, disponível e mínima de peças/insumos.
Movimentação de estoque	Registro de entrada, saída, venda, reserva confirmada ou ajuste.
Orçamento	Composição financeira da OS, calculada por serviços e peças.
Aprovação	Aceite do cliente para execução do orçamento.
Baixa de estoque	Redução definitiva do estoque após aprovação ou saída registrada.

16. Segurança implementada

Controles de segurança implementados:

- Autenticação JWT por POST /api/v1/auth/login.
- Token JWT assinado com segredo vindo de variável de ambiente.
- Expiração configurável por JWT_EXPIRATION_MINUTES.
- APIs administrativas protegidas por Bearer Token.
- Autorização centralizada por Spring Security.
- Senhas com BCrypt.
- DTOs explícitos para requests/responses.
- Rejeição de campos desconhecidos via Jackson.
- Validação real de CPF e CNPJ.
- Validação de placa brasileira antiga e Mercosul.
- Validação de e-mail, tamanhos máximos, preços e quantidades não negativos.
- Tratamento global de exceções REST.
- Erros padronizados sem retorno intencional de stacktrace ao usuário.
- CORS configurável e sem wildcard.
- Swagger desabilitável por variável de ambiente.
- Uso de .env.example sem secrets reais.

17. Testes e cobertura

O projeto possui testes unitários e de integração para regras de domínio, use cases, segurança, autorização e fluxos REST principais.

Comandos:

```
mvn test
mvn verify
```

Relatório JaCoCo:

```
target/site/jacoco/index.html
target/site/jacoco/jacoco.csv
```

Cobertura documentada no relatório de segurança, considerando a configuração Maven que ignora classes geradas pelo OpenAPI e componentes fora do escopo de negócio. Além das classes geradas, a configuração exclui a camada REST, a infraestrutura e records auxiliares de comando, consulta e saída. Portanto, os percentuais representam o núcleo de negócio medido, e não a cobertura global de todo o repositório:

Métrica	Coberto	Não coberto	Cobertura
Instruções	5.082	247	95,36%
Branches	284	78	78,45%
Linhas	1.250	34	97,35%
Métodos	348	21	94,31%
Classes	81	0	100,00%

Resultado documentado: 108 testes automatizados e `mvn verify` concluindo com sucesso.

18. Docker e execução local

Pré-requisitos:

- Java 21.
- Maven 3.9+.
- Docker.
- Docker Compose.

Criar `.env` a partir do exemplo:

```
Copy-Item .env.example .env
```

Variáveis obrigatórias mínimas:

```
POSTGRES_PASSWORD=[PREENCHER - senha local do PostgreSQL]
JWT_SECRET=[PREENCHER - segredo local com pelo menos 32 bytes]
```

Executar API e banco com Docker Compose:

```
docker compose up --build
```

Executar em background:

```
docker compose up -d --build
```

Parar ambiente:

```
docker compose down
```

Remover volume local do banco:

```
docker compose down -v
```

Executar somente PostgreSQL pelo Compose e API pelo Maven:

```
docker compose up -d postgres
```

```
mvn spring-boot:run
```

Serviços locais:

```
API: http://localhost:8080
Swagger: http://localhost:8080/swagger-ui.html
PostgreSQL: localhost:5432
```

Frontend demonstrativo:

```
cd frontend
npm install
npm run dev
```

URL padrão do frontend:

```
http://localhost:5173
```

19. Relatório de vulnerabilidades

Relatório oficial:

```
docs/SECURITY_REPORT.md
```

Relatório consolidado de validação executado em 20/06/2026:

```
docs/FINAL_VALIDATION_REPORT.md
```

Guia de execução de scans:

```
docs/SECURITY_SCAN_GUIDE.md
```

Evidências documentadas:

```
target/dependency-check/dependency-check-report.html
target/dependency-check/dependency-check-report.json
security-reports/frontend-dependencies/npm-audit-report.json
target/site/jacoco/index.html
target/site/jacoco/jacoco.csv
```

Resumo dos scans documentados:

- OWASP Dependency-Check backend: scan final com 0 vulnerabilidades reportadas.
- npm audit frontend: scan final com 0 vulnerabilidades reportadas.
- Docker Scout da imagem final: 0 vulnerabilidades.
- Gitleaks: 0 leaks em 35 commits.
- Semgrep: 0 achados e 0 erros em 190 arquivos.

20. Vulnerabilidades encontradas

As vulnerabilidades encontradas nos scans iniciais foram registradas no relatório de segurança e corrigidas antes do resultado final.

ID	Ferramenta	Severidade	Pacote/área afetada	Status
VULN-001	OWASP Dependency-Check	Alta/Média	log4j-api-2.24.3.jar	Corrigido
VULN-002	OWASP Dependency-Check	Alta	postgresql-42.7.10.jar	Corrigido
VULN-003	OWASP Dependency-Check	Crítica/Alta/Média	spring-boot-3.5.13.jar e starters	Corrigido
VULN-004	OWASP Dependency-Check	Alta/Média/Baixa	spring-core-6.2.17.jar e spring-web-6.2.17.jar	Corrigido
VULN-005	OWASP Dependency-Check	Alta/Média/Baixa	spring-security-core-6.5.9.jar e spring-security-web-6.5.9.jar	Corrigido

ID	Ferramenta	Severidade	Pacote/área afetada	Status
VULN-006	OWASP Dependency-Check	Crítica/Alta/Média/Baixa	tomcat-embed-core-10.1.53.jar	Corrigido
VULN-007	OWASP Dependency-Check	Média	commons-compress-1.24.0.jar	Corrigido
VULN-008	OWASP Dependency-Check	Média	commons-lang3-3.17.0.jar	Corrigido
VULN-009	OWASP Dependency-Check	Média/Desconhecida	swagger-ui-5.32.2.jar	Corrigido
VULN-010	npm audit	Alta	vite, esbuild, @vitejs/plugin-vue	Corrigido
VULN-011	npm audit	Média	js-yaml-4.1.1 transitivo do ESLint	Corrigido
VULN-012	Docker Scout	Crítica/Alta/Média	/usr/bin/pebble da imagem runtime anterior	Corrigido

Não há vulnerabilidades ou leaks pendentes no escopo do OWASP Dependency-Check, npm audit, Docker Scout, Gitleaks e Semgrep após os scans finais documentados.

21. Correções aplicadas

Correções aplicadas:

- Atualização do BOM Spring Boot para 4.1.0.
- Atualização transitiva do Spring Framework para 7.0.8.
- Atualização transitiva do Spring Security para 7.1.0.
- Atualização transitiva do Tomcat para 11.
- Atualização do Log4j API para 2.25.4.
- Atualização do PostgreSQL JDBC para 42.7.11.
- Override para Commons Compress 1.28.0.
- Atualização transitiva do Commons Lang para 3.20.0.
- Atualização direta do Swagger UI para 5.32.6.
- Atualização do frontend para vite 8.0.16 e @vitejs/plugin-vue 6.0.7.
- Atualização transitiva do js-yaml via npm audit fix.
- Regeneração do package-lock.json.
- Migração da imagem runtime para gcr.io/distroless/java21-debian12:nonroot.
- Reexecução dos scans finais com 0 vulnerabilidades reportadas para backend, frontend e imagem Docker.

Nenhuma vulnerabilidade foi formalmente aceita como risco na análise documentada.

22. Limitações conhecidas

Limitações do MVP:

- Não há pagamento online.
- Não há envio real de e-mail, SMS ou WhatsApp.
- Histórico de status da OS é simplificado.
- Controle de múltiplas oficinas/lojas existe de forma simplificada no mesmo monolito.
- Swagger fica público no ambiente local acadêmico.
- Não há event store para eventos de domínio.

- Um teste dinâmico dedicado de segurança permanece como melhoria futura.
- A cobertura pode ser ampliada em controllers, adapters e fluxos negativos.
- Frontend é demonstrativo e complementar ao backend.

23. Melhorias futuras

Melhorias planejadas ou recomendadas:

- Ampliar cobertura automatizada de controllers, adapters e cenários negativos.
- Criar auditoria de ações sensíveis.
- Melhorar histórico detalhado de status da OS.
- Integrar notificações reais por e-mail, SMS ou WhatsApp.
- Restringir Swagger por ambiente/perfil em produção.
- Evoluir suporte multiempresa/multitenancy.
- Adicionar pipeline CI com build, testes, cobertura e scans de segurança.
- Reexecutar Docker Scout, Gitleaks e Semgrep em cada ciclo de entrega.
- Evoluir métricas operacionais da oficina.
- Implementar integrações externas para pagamento, agenda e comunicação.
- Avaliar implementação futura de eventos de domínio persistidos.

24. Conclusão

O AutoCare Hub entrega um MVP backend coerente com o desafio proposto, cobrindo o ciclo principal de atendimento de uma oficina mecânica: cadastro de clientes e veículos, criação de Ordem de Serviço, composição com serviços e peças, geração e aprovação de orçamento, controle de status, estoque e consulta pelo cliente.

A solução utiliza arquitetura em camadas, aplica DDD de forma pragmática, documenta Event Storming e linguagem ubíqua, disponibiliza contrato OpenAPI/Swagger, implementa autenticação JWT, valida dados sensíveis e oferece execução local com Docker.

Os testes automatizados e a análise de vulnerabilidades documentada reforçam a qualidade da entrega. As vulnerabilidades identificadas nos scans iniciais de dependências foram corrigidas, e os scans finais de backend e frontend não reportaram vulnerabilidades abertas. As limitações restantes estão registradas como escopo conhecido do MVP e como oportunidades de evolução futura.